Zomato Data Analysis Using Python

In [5]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

Creating the data frame.

In [6]:
```
dataframe = pd.read_csv("D:\data analysis\geeksforgeeks\zomato\Zomato-data-.csv")
print(dataframe.head())
```

```
                    name online_order book_table   rate  votes  \
0                  Jalsa          Yes        Yes  4.1/5    775
1         Spice Elephant          Yes         No  4.1/5    787
2        San Churro Cafe          Yes         No  3.8/5    918
3  Addhuri Udupi Bhojana           No         No  3.7/5     88
4          Grand Village           No         No  3.8/5    166

   approx_cost(for two people) listed_in(type)
0                          800          Buffet
1                          800          Buffet
2                          800          Buffet
3                          300          Buffet
4                          600          Buffet
```

data Exploration

In [7]:
```
dataframe.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 148 entries, 0 to 147
Data columns (total 7 columns):
 #   Column                       Non-Null Count  Dtype
---  ------                       --------------  -----
 0   name                         148 non-null    object
 1   online_order                 148 non-null    object
 2   book_table                   148 non-null    object
 3   rate                         148 non-null    object
 4   votes                        148 non-null    int64
 5   approx_cost(for two people)  148 non-null    int64
 6   listed_in(type)              148 non-null    object
dtypes: int64(2), object(5)
memory usage: 8.2+ KB
```

In [8]:
```
print(dataframe.isnull().sum())
```

```
name                           0
online_order                   0
book_table                     0
rate                           0
votes                          0
approx_cost(for two people)    0
listed_in(type)                0
dtype: int64
```

preparation

In [9]:
```
def handleRate(value):
    value=str(value).split('/')
    value=value[0];
    return float(value)

dataframe['rate'] = dataframe['rate'].apply(handleRate)
print(dataframe.head())
```

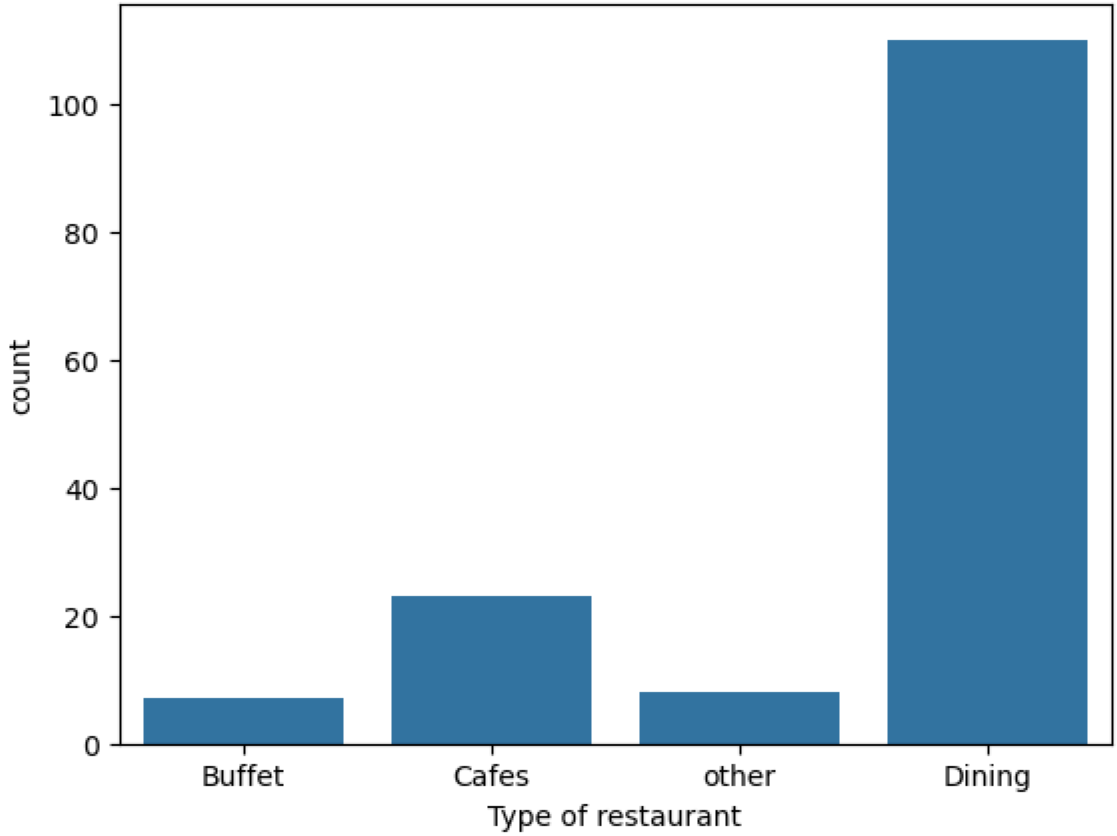
```
                    name online_order book_table  rate  votes  \
0                  Jalsa          Yes        Yes   4.1    775
1         Spice Elephant          Yes         No   4.1    787
2        San Churro Cafe          Yes         No   3.8    918
3  Addhuri Udupi Bhojana           No         No   3.7     88
4          Grand Village           No         No   3.8    166

   approx_cost(for two people) listed_in(type)
0                          800          Buffet
1                          800          Buffet
2                          800          Buffet
3                          300          Buffet
4                          600          Buffet
```

1-Exploring Restaurant Types

In [10]:
```
sns.countplot(x=dataframe['listed_in(type)'])
plt.xlabel('Type of restaurant')
```

Out[10]:
```
Text(0.5, 0, 'Type of restaurant')
```



🔍 Insight_1: that the majority of restaurants fall under dining category

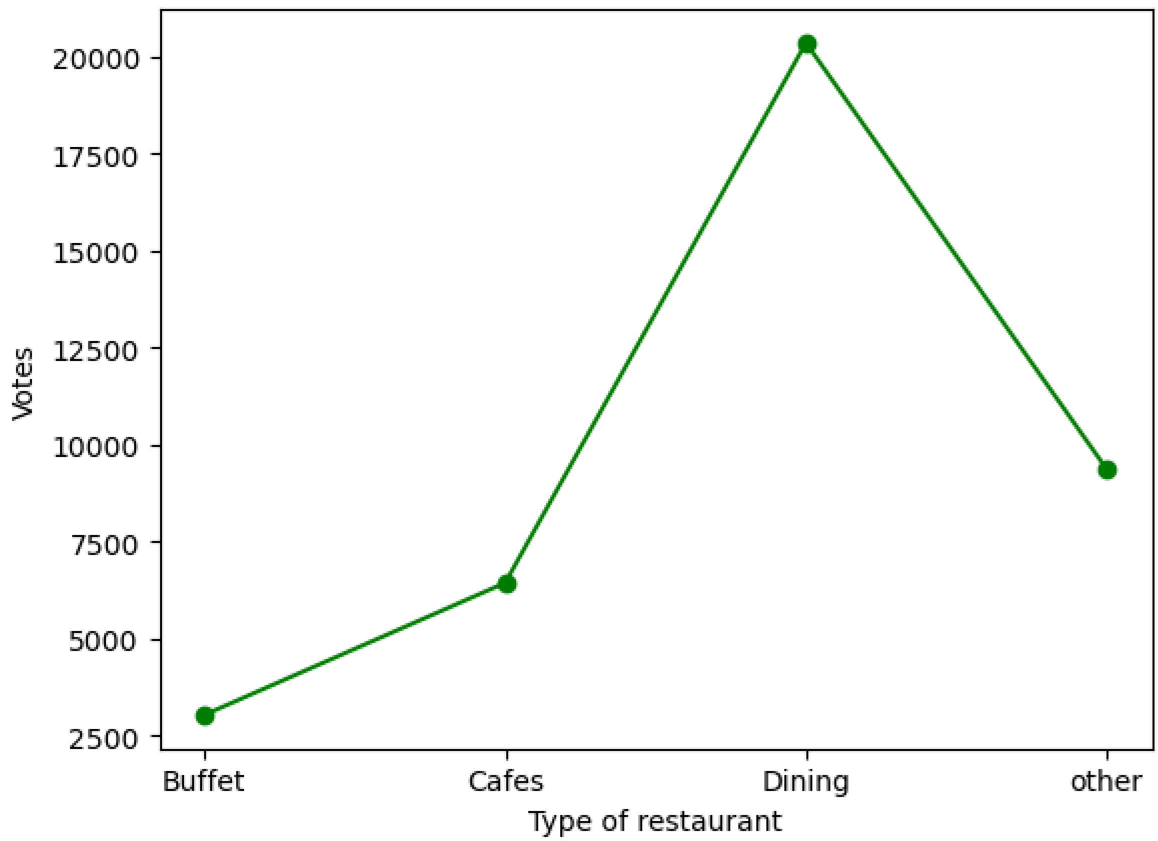
- Dominance: Dining restaurants completely control the market with over 110 establishments, reflecting a highly traditional market and fierce competition within this category.

- Investment Opportunity: The significant shortage of buffet restaurants and cafes represents a promising market gap, allowing new investors to penetrate the market without being caught in the crossfire of competition.

2- Votes by Restaurant Type

In [11]:
```
grouped_data = dataframe.groupby('listed_in(type)')['votes'].sum()
result = pd.DataFrame({'votes': grouped_data})
plt.plot(result, c='green', marker='o')
plt.xlabel('Type of restaurant')
plt.ylabel('Votes')
```

Out[11]:
```
Text(0, 0.5, 'Votes')
```



🔍 insight_2:Dining restaurants are preferred by a larger number of individuals.

-Dominance in Engagement: Dining restaurants dominate the overall engagement in the market with over 20,000 votes, confirming their position as the category that most attracts consumer attention and evaluation.

-Lack of Momentum: Buffet and Cafe categories exhibit virtually no engagement or momentum (less than 5,000 votes), a natural consequence of their smaller market presence compared to Dining.

3-Identify the Most Voted Restaurant

In [12]:
```
max_votes = dataframe['votes'].max()
restaurant_with_max_votes = dataframe.loc[dataframe['votes'] == max_votes,'name']

print('Restaurant(s) with the maximum votes:')
print(restaurant_with_max_votes)
```

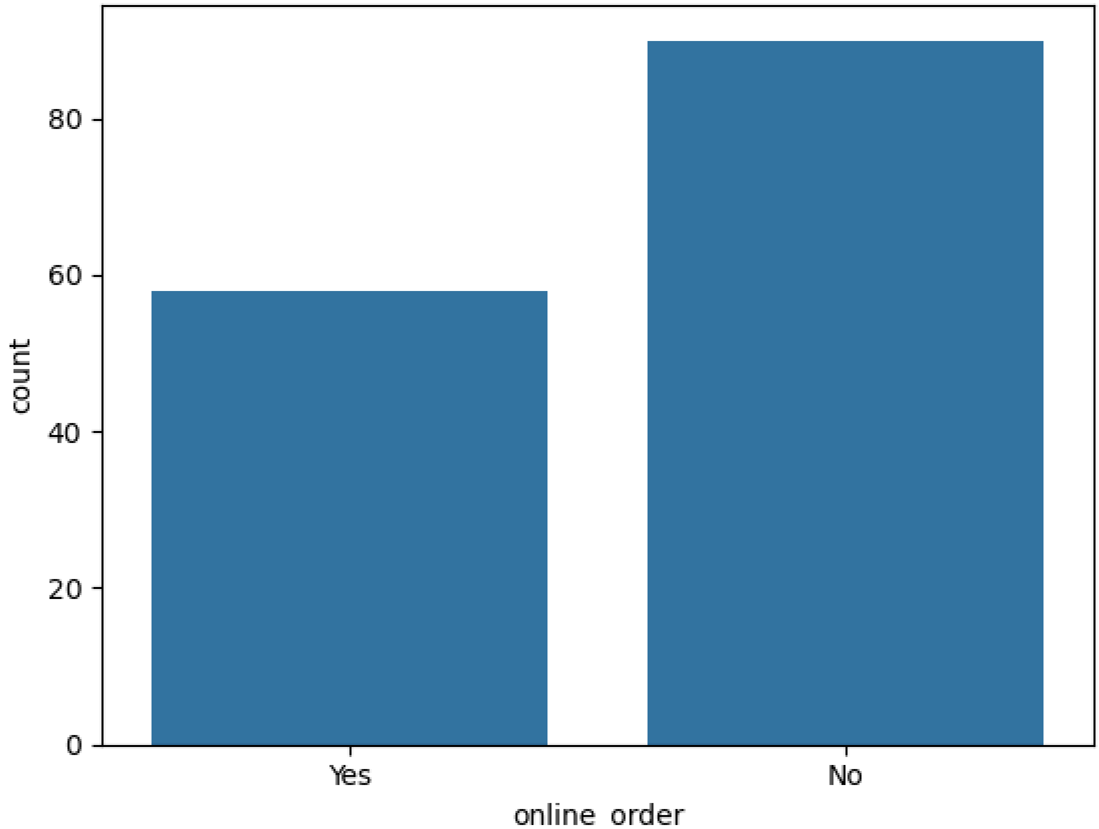
```
Restaurant(s) with the maximum votes:
38    Empire Restaurant
Name: name, dtype: object
```

🔍 insight_3:The restaurant that received the highest number of votes is Empire Restaurant.

4- Online Order Availability

In [13]:
```
sns.countplot(x=dataframe['online_order'])
```

Out[13]: <Axes: xlabel='online_order', ylabel='count'>



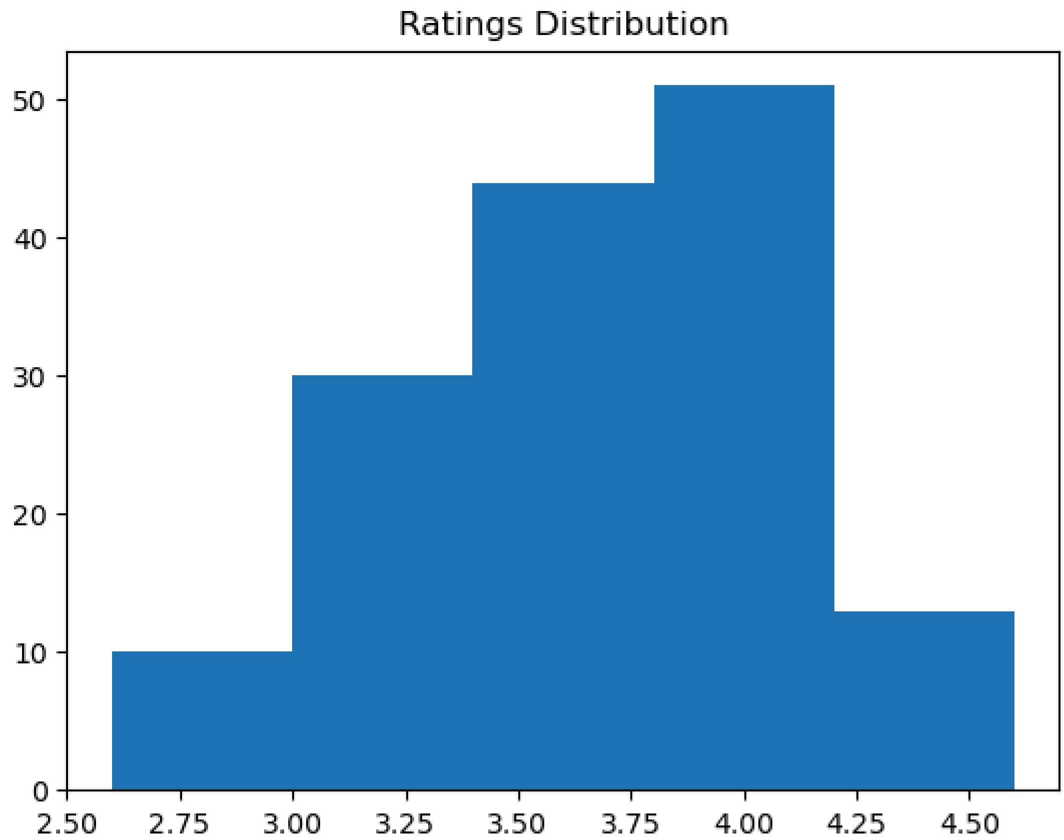
🔍 insight_4: majority of the restaurants do not accept online orders.

-Digital Transformation Gap: The majority of restaurants in the market (approximately 90) prefer not to activate online ordering (No), meaning they rely on in-person customer visits.

-Competitive Advantage Opportunity: This situation could present an opportunity for investors to penetrate the market and attract a broad consumer base by offering an online ordering and delivery system.

5-Analyze Ratings

In [14]:
```
plt.hist(dataframe['rate'],bins=5)
plt.title('Ratings Distribution')
plt.show()
```



🔍 insight_5:The majority of restaurants received ratings ranging from 3.3 to 4.1.

-High quality and competitiveness: The majority of restaurants fall within the highly positive range of 3.3 to 4.1 stars (peaking at over 50 restaurants), indicating that the market offers a generally satisfactory dining experience.

-A scarcity of poor ratings: The number of restaurants with poor ratings (below 3.0) is significantly lower.

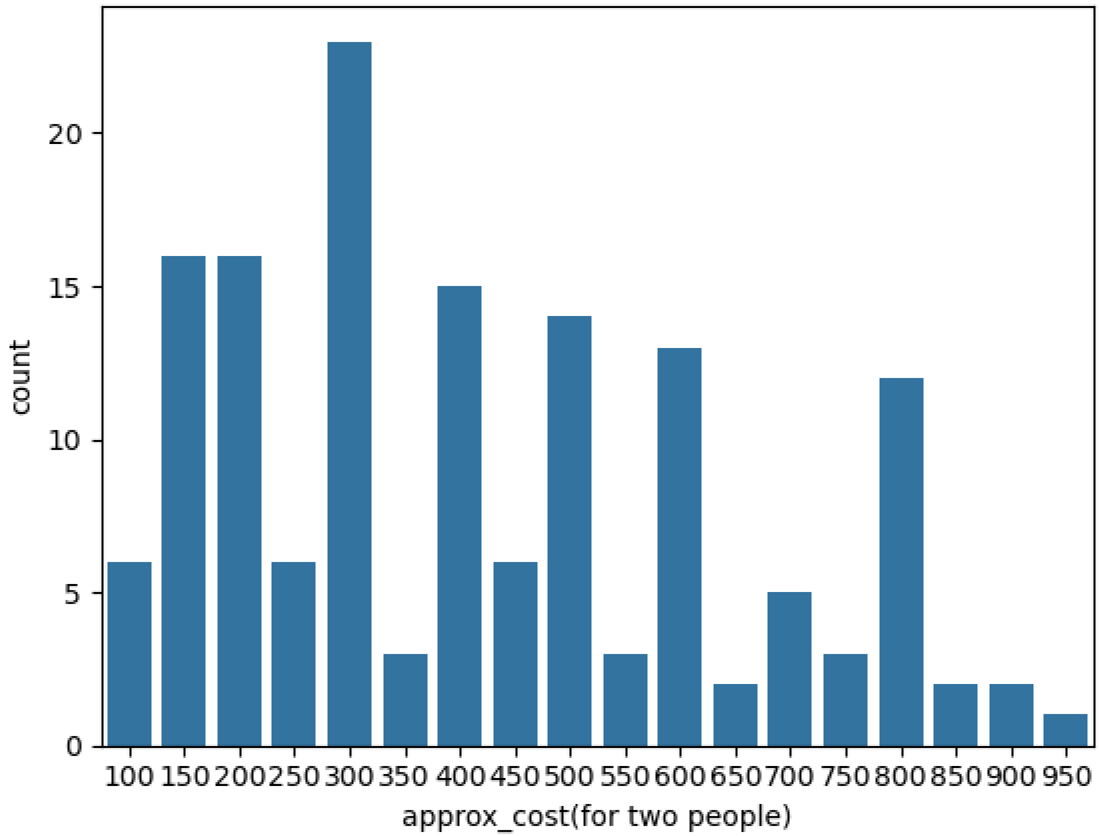
6-Approximate Cost for Couples

In [15]:

```
couple_data = dataframe['approx_cost(for two people)']
sns.countplot(x=couple_data )
```

Out[15]:

```
<Axes: xlabel='approx_cost(for two people)', ylabel='count'>
```



🔍 insight_6:The majority of couples prefer restaurants with an approximate cost of 300 $.

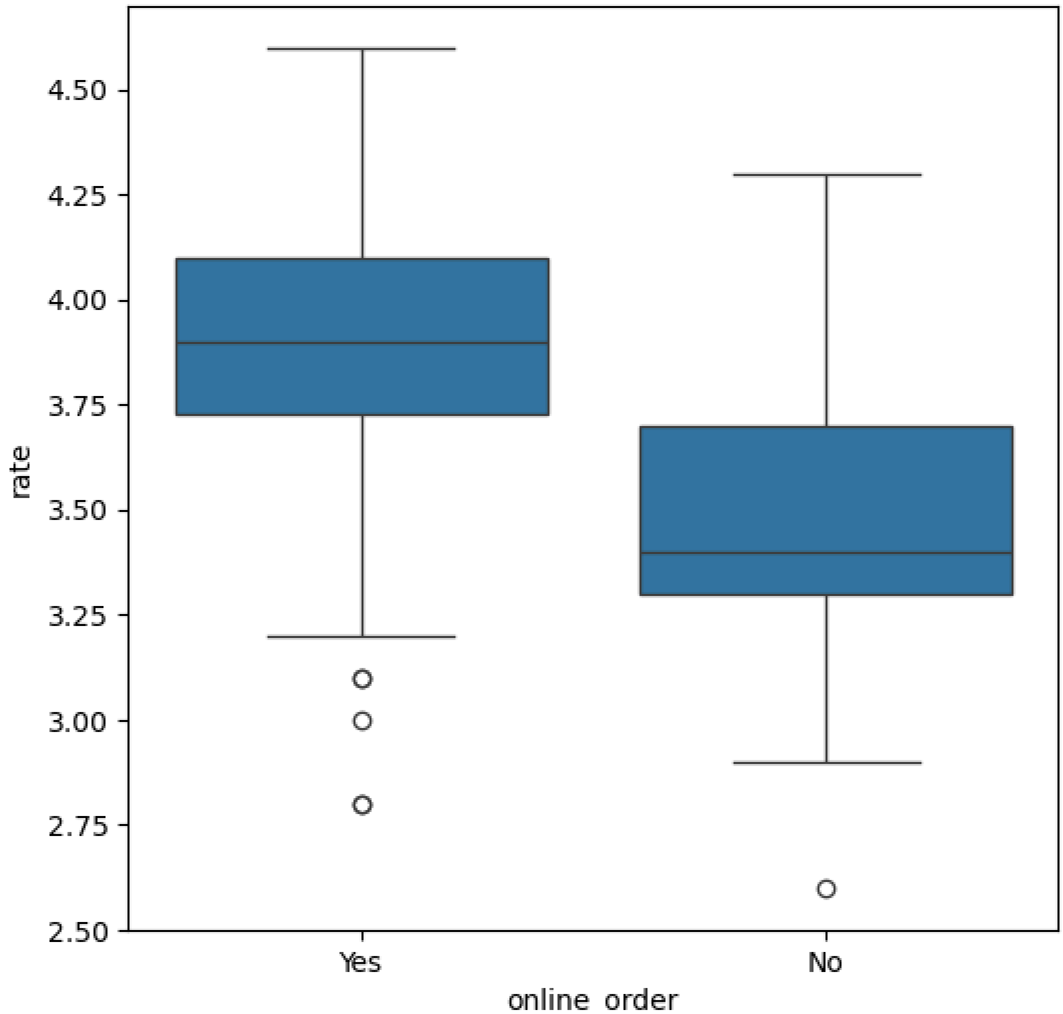
-Mid-range concentration: The most frequent value for restaurants is at 300 USD for two people (with more than 20 restaurants), which reflects the dominance of the economy and mid-range price category over most market options.

-Scarcity of luxury options: The number of restaurants decreases sharply as the cost rises above $600, indicating that the market is primarily for consumers looking for value for price rather than pure luxury (premium segment).

7-Ratings Comparison - Online vs Offline Orders

In [17]:
```
plt.figure(figsize=(6,6))
sns.boxplot(x= 'online_order',y='rate',data= dataframe)
```

Out[17]: `<Axes: xlabel='online_order', ylabel='rate'>`



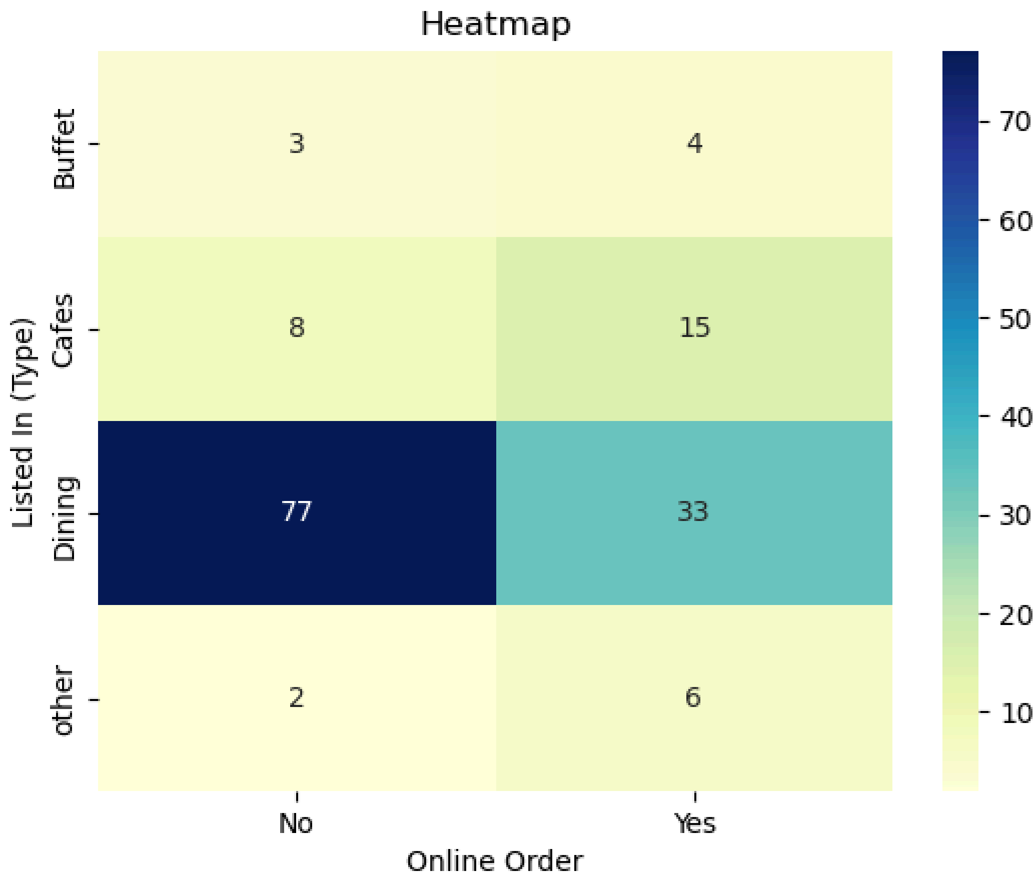
🔍 insight_7:Offline orders received lower ratings in comparison to online orders which obtained excellent ratings.

-Clear numerical advantage: Restaurants that offer online ordering (Yes) achieve significantly higher ratings (median close to 3.9) compared to traditional restaurants (No), whose median rating stands at approximately 3.4, reflecting customer satisfaction with the online shopping experience.

-Consistent performance: The "Yes" option falls entirely within a narrow, high rating range (between 3.75 and 4.1), indicating consistent online service quality, while traditional restaurants suffer from inconsistency and a clear decline in overall ratings.

8-Order Mode Preferences by Restaurant Type

In [18]:
```
pivot_table = dataframe.pivot_table(index='listed_in(type)', columns= 'online_order
sns.heatmap(pivot_table, annot=True, cmap='YlGnBu', fmt='d')
plt.title('Heatmap')
plt.xlabel('Online Order')
plt.ylabel('Listed In (Type)')
plt.show()
```



🔍 insight_8:

1.Dining restaurants dominate the market with a total of 110 establishments.

2.Most dining restaurants (70%) refuse online ordering.

3.Cafes rely heavily on online services; twice as many offer them as prohibit them.

```
In [ ]:
```